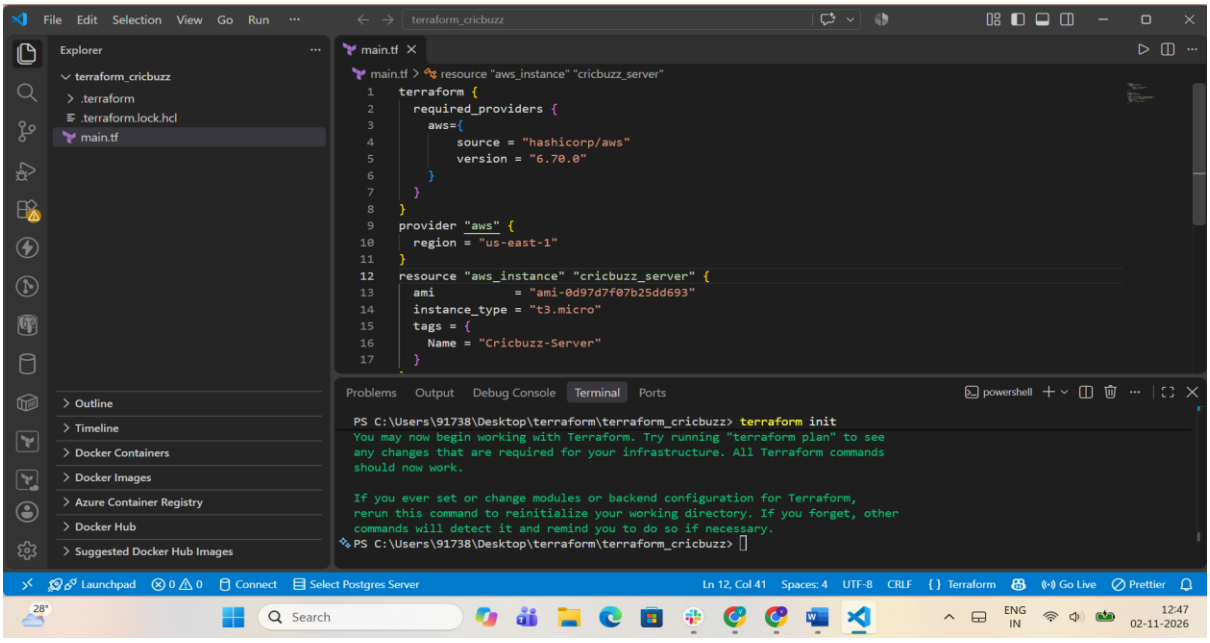
Terraform Challenges

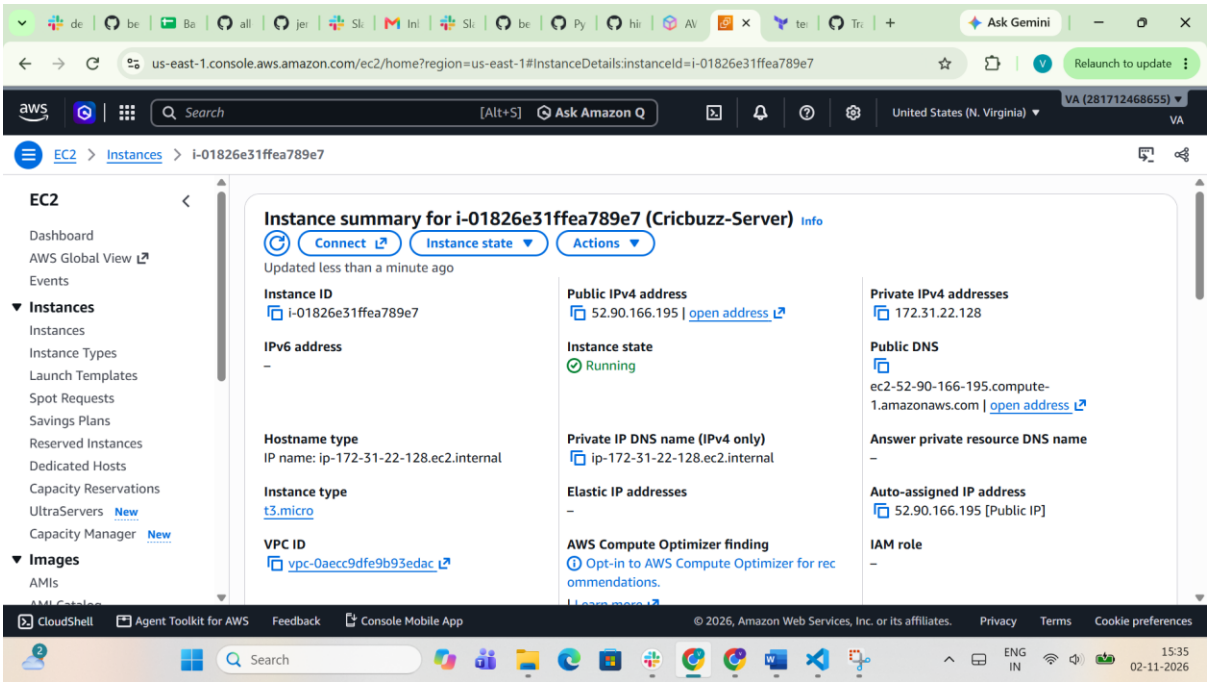
1. **Create an EC2 Instance on AWS Cloud using the below-mentioned ubuntu AMI and instance_type. Or, (You can use of your choice as well) ami = "ami-0d97d7f07b25dd693" instance_type = "t3.micro"**

   Implementations steps:

   1. Configured the AWS provider with the required region.
   2. Defined the Ubuntu AMI ID and EC2 instance type.
   3. Created an EC2 instance using the Terraform aws_instance resource.
   4. Assigned the name tag Cricbuzz-Server.
   5. Initialized and validated the Terraform configuration.
   6. Executed terraform apply to provision the EC2 instance.
   7. Verified the instance status in the AWS EC2 Console.




```
provider "aws" {
  region = "us-east-1"
}
resource "aws_instance" "cricbuzz_server" {
  ami           = "ami-0d97d7f07b25dd693"
  instance_type = "t3.micro"
  key_name      = "linux-testing"
  vpc_security_group_ids = [aws_security_group.web_sg.id]
  tags = {
    Name = "Cricbuzz-Server"
  }
}
```



2. Install a web server on the same EC2 Instance. The web server should listen on port 8080.

Implementations steps:

1. Create a shell script named install_nginx.sh in the Terraform project folder.
2. Add commands to update the Ubuntu package repository.
3. Install the Nginx package.
4. Enable the Nginx service to start automatically when the instance boots.
5. Configure the Nginx default site to listen on port 8080.
6. Test the Nginx configuration.
7. Restart the Nginx service to apply the changes.
8. Create a Terraform null_resource to execute the shell script on the existing EC2 instance.
9. Use the Terraform file provisioner to copy the shell script to the EC2 instance.
10. Use the remote-exec provisioner to execute the script.
11. Run terraform plan and terraform apply.

Shell Script (install_nginx.sh)

```
#!/bin/bash

set -e
```

```
# Update package repository
sudo apt-get update -y

# Install Nginx
sudo apt-get install -y nginx

# Configure Nginx to listen on port 8080
sudo tee /etc/nginx/sites-available/default > /dev/null <<'EOF'
server {
    listen 8080 default_server;
    listen [::]:8080 default_server;
    root /var/www/html;
    index index.html;
    server_name _;
    location / {
        try_files $uri $uri/ =404;
    }
}
EOF

# Test Nginx configuration
sudo nginx -t

# Enable and restart Nginx
sudo systemctl enable nginx
sudo systemctl restart nginx

echo "Nginx installed and configured on port 8080 successfully!"
```

Terraform Code (main.tf)

Add the following resource to the existing main.tf file. Do not create another EC2 instance.

```
resource "null_resource" "install_nginx" {
  triggers = {
    instance_id = aws_instance.cricbuzz_server.id
    version     = "1"
  }

  connection {
    type        = "ssh"
    user        = "ubuntu"
    host        = aws_instance.cricbuzz_server.public_ip
    private_key = file("${path.module}/linux-testing.pem")
    timeout     = "5m"
  }

  provisioner "file" {
    source      = "${path.module}/install_nginx.sh"
    destination = "/tmp/install_nginx.sh"
  }

  provisioner "remote-exec" {
    inline = [
      "chmod +x /tmp/install_nginx.sh",
      "sudo bash /tmp/install_nginx.sh"
    ]
  }
}
```

Terraform Commands

terraform init

terraform plan

terraform apply

Verification

1. Connect to the EC2 instance using SSH.
2. Check the Nginx service status:

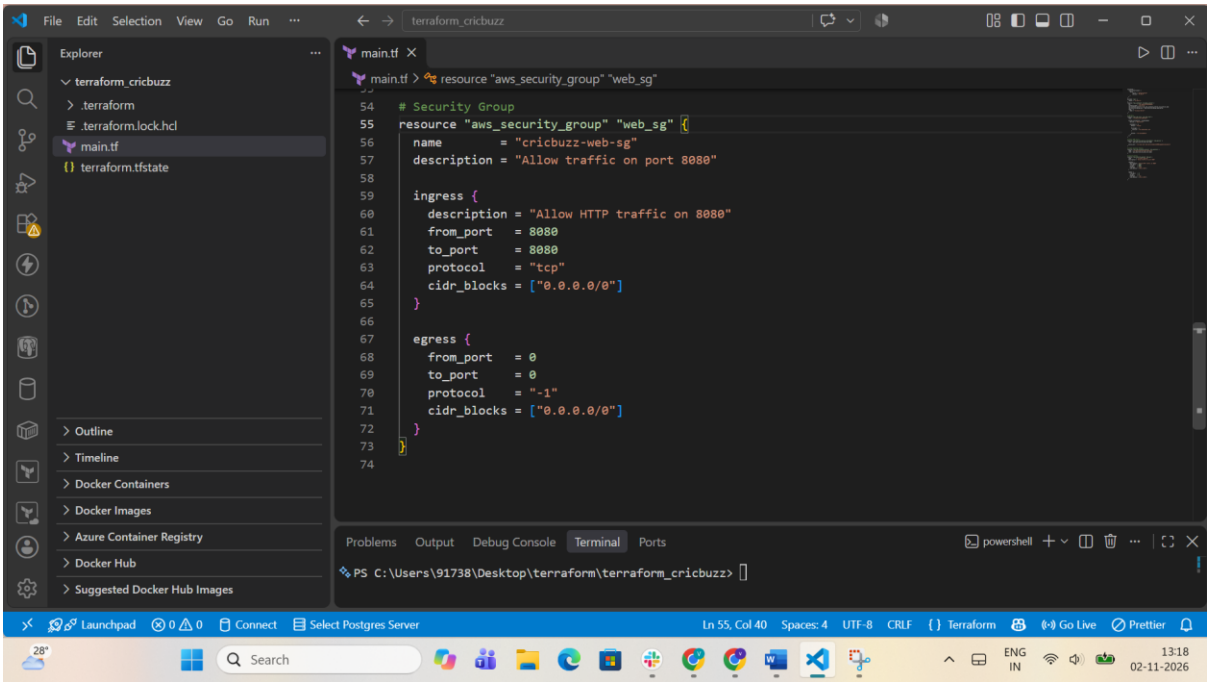
   sudo systemctl status nginx
3. Verify that Nginx is listening on port 8080:

   sudo ss -tulnp | grep 8080
4. Test Nginx locally:

   curl http://localhost:8080
5. Open the following URL in a browser:

   http://<EC2-PUBLIC-IP>:8080

   The Nginx welcome page should be displayed.





```
$ install_nginx.sh
#!/bin/bash
set -e

# Update package repository
sudo apt-get update -y

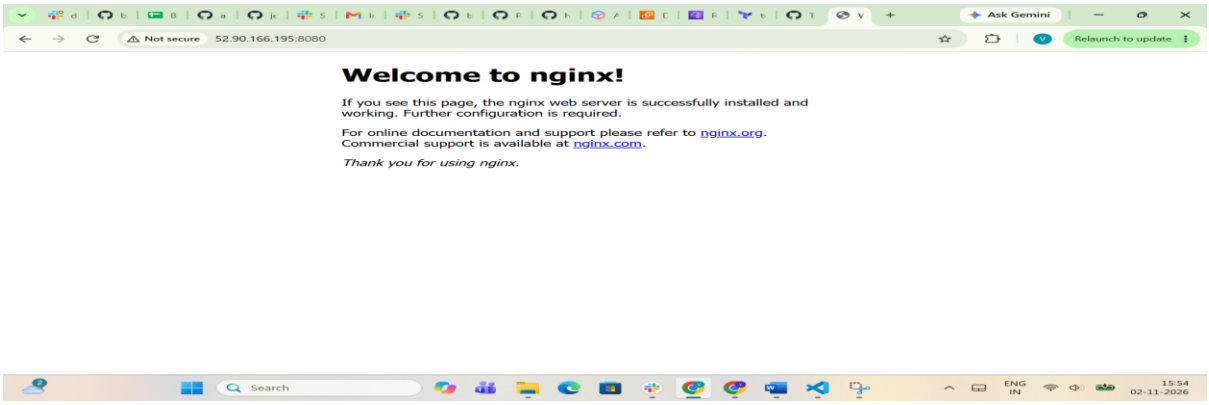
# Install Nginx
sudo apt-get install -y nginx

# Configure Nginx to listen on port 8080
sudo tee /etc/nginx/sites-available/default > /dev/null <<'EOF'
server {
    listen 8080 default_server;
    listen [::]:8080 default_server;

    root /var/www/html;
    index index.html;

    server_name _;

    location / {
        try_files $uri $uri/ =404;
```



3. **Create a Docker-compose file for the project: https://github.com/betawins/cricbuzz-docker-hello-world.git and point it to port 8000.**

Implementations steps:

1. Create a shell script named deploy_cricbuzz.sh in the Terraform project folder.
2. Update the Ubuntu package repository.
3. Install Git, Docker, and Go.
4. Enable and start the Docker service.
5. Clone the Cricbuzz application repository from GitHub.
6. Navigate to the cloned repository.
7. Compile the Go application into a Linux executable binary named hello_world.
8. Create a docker-compose.yml file.

9. Configure Docker Compose to build the image using the repository's Dockerfile.
10. Map host port 8000 to container port 8080.
11. Stop any previously running application containers.
12. Build the Docker image and start the application using Docker Compose.
13. Create a Terraform null_resource to copy and execute the shell script on the existing EC2 instance.
14. Run Terraform plan and apply.

Shell Script (deploy_cricbuzz.sh)

```
#!/bin/bash
set -e

# Update package repository
sudo apt-get update -y

# Install required packages
sudo apt-get install -y git docker.io golang-go

# Start and enable Docker
sudo systemctl enable docker
sudo systemctl start docker

# Clone the GitHub repository
cd /home/ubuntu

if [ ! -d "cricbuzz-docker-hello-world" ]; then
   git clone https://github.com/betawins/cricbuzz-docker-hello-world.git
fi

# Navigate to application directory
cd /home/ubuntu/cricbuzz-docker-hello-world

# Compile the Go application
CGO_ENABLED=0 GOOS=linux GOARCH=amd64 go build -a -installsuffix cgo -o
hello_world .

# Create Docker Compose file
sudo tee docker-compose.yml > /dev/null <<'EOF'
services:
  cricbuzz:
    build: .
    container_name: cricbuzz-app
    ports:
      - "8000:8080"
    restart: always
```

```
EOF

# Stop existing containers and deploy the application
sudo docker compose down || true
sudo docker compose up -d --build

echo "Cricbuzz application deployed successfully!"
```

Terraform Code (main.tf)

Add the following resource to the existing main.tf file. Do not create another EC2 instance.

```
resource "null_resource" "deploy_cricbuzz" {
 triggers = {
  instance_id = aws_instance.cricbuzz_server.id
  version     = "3"
 }

 connection {
  type        = "ssh"
  user        = "ubuntu"
  host        = aws_instance.cricbuzz_server.public_ip
  private_key = file("${path.module}/linux-testing.pem")
  timeout     = "5m"
 }

 provisioner "file" {
  source      = "${path.module}/deploy_cricbuzz.sh"
  destination = "/tmp/deploy_cricbuzz.sh"
 }

 provisioner "remote-exec" {
  inline = [
   "chmod +x /tmp/deploy_cricbuzz.sh",
   "sudo bash /tmp/deploy_cricbuzz.sh"
  ]
 }
}
```

Terraform Commands

```
terraform init
terraform plan
terraform apply
```

Verification

1. Connect to the EC2 instance using SSH.
2. Check whether the application container is running:
   sudo docker ps
3. Check the application container logs:
   sudo docker logs cricbuzz-app
4. Test the application locally:
   curl http://localhost:8000/
   Expected output:
   Hello, World!
5. Open the following URL in a browser:
   http://<EC2-PUBLIC-IP>:8000
   The Cricbuzz application should respond successfully.

deploy_cricbuzz.sh

```
#!/bin/bash
set -e

# Update package repository
sudo apt-get update -y

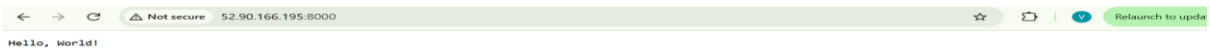
# Install required packages
sudo apt-get install -y git docker.io golang-go

# Start and enable Docker
sudo systemctl enable docker
sudo systemctl start docker

# Clone the GitHub repository
cd /home/ubuntu

if [ ! -d "cricbuzz-docker-hello-world" ]; then
    git clone https://github.com/betawins/cricbuzz-docker-hello-world.git
fi

# Navigate to application directory
cd /home/ubuntu/cricbuzz-docker-hello-world
```



4. Make the web server point to the above service so that 8080 returns the response from the Docker service.

Implementations steps:

1. Create a shell script named configure_nginx.sh in the Terraform project folder.
2. Update the Nginx default site configuration.
3. Configure Nginx to listen on port 8080.
4. Add a reverse proxy configuration using proxy_pass to forward requests to http://127.0.0.1:8000.
5. Configure the required proxy headers to forward client request information.
6. Test the Nginx configuration using nginx -t.
7. Enable and restart the Nginx service.
8. Create a Terraform null_resource to copy and execute the shell script on the existing EC2 instance.
9. Run Terraform plan and apply.

Shell Script (configure_nginx.sh)

```
#!/bin/bash
set -e

# Configure Nginx as a reverse proxy
sudo tee /etc/nginx/sites-available/default > /dev/null <<'EOF'
server {
    listen 8080 default_server;
    listen [::]:8080 default_server;

    server_name _;

    location / {
        proxy_pass http://127.0.0.1:8000;

        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
EOF

# Test Nginx configuration
sudo nginx -t

# Enable and restart Nginx
sudo systemctl enable nginx
```

```
sudo systemctl restart nginx

echo "Nginx reverse proxy configured successfully!"
```

Terraform Code (main.tf)

Add the following resource to the existing main.tf file. Do not create another EC2 instance.

```
resource "null_resource" "install_nginx" {
 triggers = {
  instance_id = aws_instance.cricbuzz_server.id
  version     = "2"
 }

 connection {
  type        = "ssh"
  user        = "ubuntu"
  host        = aws_instance.cricbuzz_server.public_ip
  private_key = file("${path.module}/linux-testing.pem")
  timeout     = "5m"
 }

 provisioner "file" {
  source      = "${path.module}/configure_nginx.sh"
  destination = "/tmp/configure_nginx.sh"
 }

 provisioner "remote-exec" {
  inline = [
   "chmod +x /tmp/configure_nginx.sh",
   "sudo bash /tmp/configure_nginx.sh"
  ]
 }
}
```

Terraform Commands

```
terraform plan
terraform apply
```

Verification

1. Check the Nginx service status:
   `sudo systemctl status nginx`
2. Verify that Nginx is listening on port 8080:
   `sudo ss -tulnp | grep 8080`
3. Test the application through Nginx:

curl http://localhost:8080/

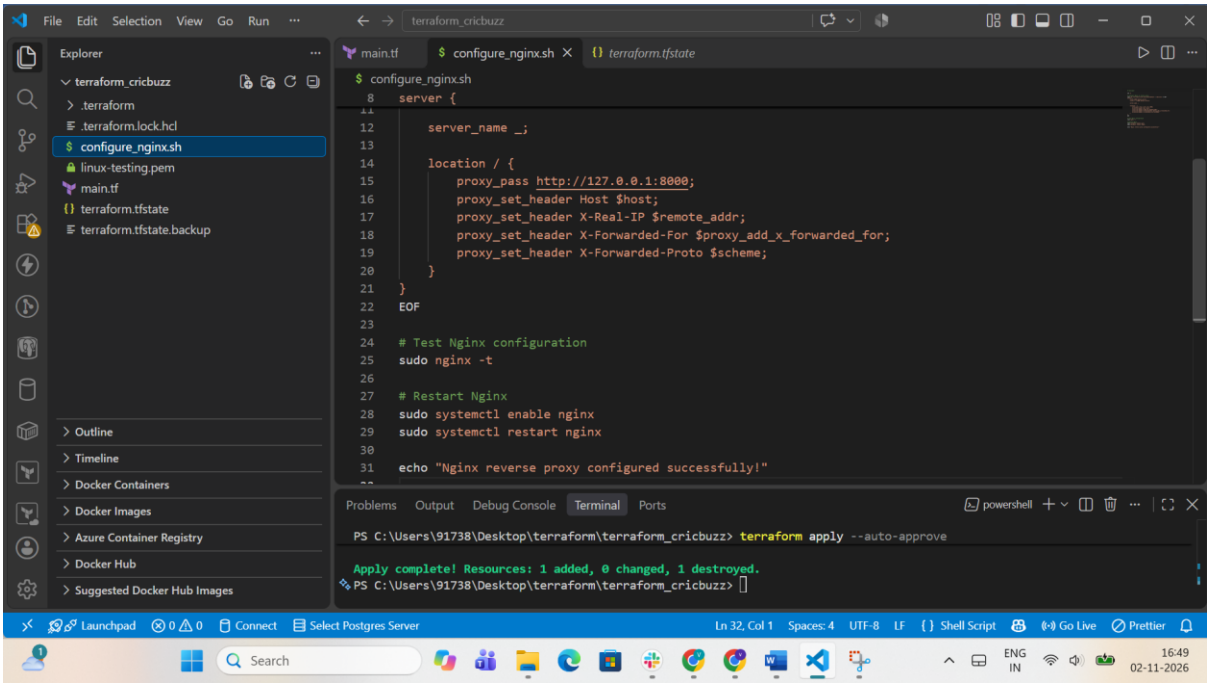
Expected output:

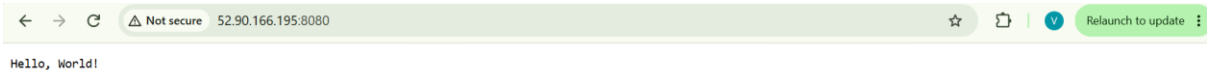
Hello, World!

4. Open the following URL in a browser:
   http://<EC2-PUBLIC-IP>:8080
   The Cricbuzz application should be displayed through Nginx.

Request Flow

Browser → EC2 Port 8080 → Nginx Reverse Proxy → Docker Application Port 8000







5. Configure to Save the Logs in the location /var/log/cb.log

Implementations steps:

1. Create a shell script named configure_access_logs.sh in the Terraform project folder.
2. Update the Nginx default site configuration.
3. Add the access_log directive inside the Nginx server block.
4. Specify /var/log/cb.log as the access log file path.
5. Test the Nginx configuration using nginx -t.
6. Restart the Nginx service to apply the changes.
7. Create a Terraform null_resource to copy and execute the shell script on the existing EC2 instance.
8. Run Terraform plan and apply.
9. Generate HTTP requests to the application to create access log entries.
10. Verify that the requests are recorded in /var/log/cb.log.

Shell Script (configure_access_logs.sh)

```
#!/bin/bash

set -e


# Configure Nginx access log file

sudo sed -i '/server_name _;/a\    access_log /var/log/cb.log;' /etc/nginx/sites-available/default


# Test Nginx configuration

sudo nginx -t


# Restart Nginx

sudo systemctl restart nginx


echo "Nginx access logging configured successfully!"
```

Terraform Code (main.tf)

Add the following resource to the existing main.tf file.

```
resource "null_resource" "configure_access_logs" {
```

```
  triggers = {
    instance_id = aws_instance.cricbuzz_server.id
    version     = "1"
  }

  connection {
    type        = "ssh"
    user        = "ubuntu"
    host        = aws_instance.cricbuzz_server.public_ip
    private_key = file("${path.module}/linux-testing.pem")
    timeout     = "5m"
  }

  provisioner "file" {
    source      = "${path.module}/configure_access_logs.sh"
    destination = "/tmp/configure_access_logs.sh"
  }

  provisioner "remote-exec" {
    inline = [
      "chmod +x /tmp/configure_access_logs.sh",
      "sudo bash /tmp/configure_access_logs.sh"
    ]
  }
}
```

Terraform Commands

terraform plan

terraform apply

Verification

1. Check whether the access log file exists:

sudo ls -l /var/log/cb.log

2. Generate a request to the application through Nginx:

   curl http://localhost:8080/

   Expected application output:

   Hello, World!

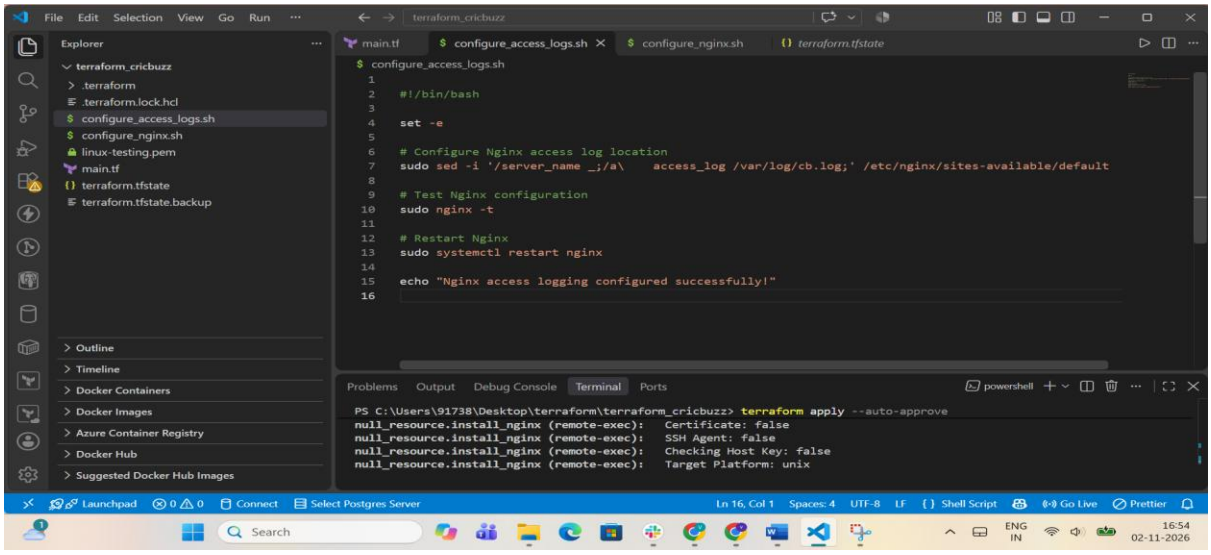
3. Check the access log entries:

   sudo tail -f /var/log/cb.log

   The log file should contain the HTTP request details, including the client IP address, request method, requested URL, HTTP status code, and response size.

   Example:

   103.203.172.230 - - [02/Nov/2026:11:32:03 +0000] "GET / HTTP/1.1" 200 13







6. Write the Log rotate configuration for the above file

Implementations steps:

1. Create a shell script named logrotate_cb.sh in the Terraform project folder.
2. Create a logrotate configuration file at /etc/logrotate.d/cb.
3. Specify /var/log/cb.log as the log file to rotate.
4. Configure daily log rotation.
5. Set the retention period to seven rotated log files.
6. Enable compression for rotated log files.
7. Configure delayed compression using delaycompress.
8. Add missingok to avoid errors if the log file is missing.
9. Add notifempty to prevent rotation of empty log files.
10. Configure the ownership and permissions of newly created log files.
11. Add the su directive to specify the user and group used during rotation.
12. Create a Terraform null_resource to copy and execute the shell script on the existing EC2 instance.
13. Run Terraform plan and apply.
14. Verify the logrotate configuration using debug mode.
15. Force log rotation to confirm that the configuration works.

Shell Script (logrotate_cb.sh)

```
#!/bin/bash
set -e

# Create logrotate configuration
sudo tee /etc/logrotate.d/cb > /dev/null <<'EOF'
/var/log/cb.log {
    daily
    rotate 7
    compress
    delaycompress
    missingok
    notifempty
    create 0644 root root
    su root adm
}
EOF

echo "Logrotate configuration created successfully!"
```

Terraform Code (main.tf)

Add the following resource to the existing main.tf file.

```
resource "null_resource" "configure_logrotate" {
  triggers = {
    instance_id = aws_instance.cricbuzz_server.id
```

```
  version     = "2"
 }

 connection {
  type        = "ssh"
  user        = "ubuntu"
  host        = aws_instance.cricbuzz_server.public_ip
  private_key = file("${path.module}/linux-testing.pem")
  timeout     = "5m"
 }

 provisioner "file" {
  source      = "${path.module}/logrotate_cb.sh"
  destination = "/tmp/logrotate_cb.sh"
 }

 provisioner "remote-exec" {
  inline = [
   "chmod +x /tmp/logrotate_cb.sh",
   "sudo bash /tmp/logrotate_cb.sh"
  ]
 }
}
```

Terraform Commands

terraform plan
terraform apply

Verification

1. Check the logrotate configuration:
   sudo cat /etc/logrotate.d/cb
2. Test the configuration in debug mode:
   sudo logrotate -d /etc/logrotate.d/cb
3. Force log rotation:
   sudo logrotate -f /etc/logrotate.d/cb
4. Verify the rotated log files:
   sudo ls -lh /var/log/cb*
   Example output after successful rotation:
   -rw-r--r-- 1 root root   0 Nov 2 11:45 /var/log/cb.log
   -rw-r--r-- 1 root root 843 Nov 2 11:33 /var/log/cb.log.1

$ logrotate_cb.sh

```
#!/bin/bash

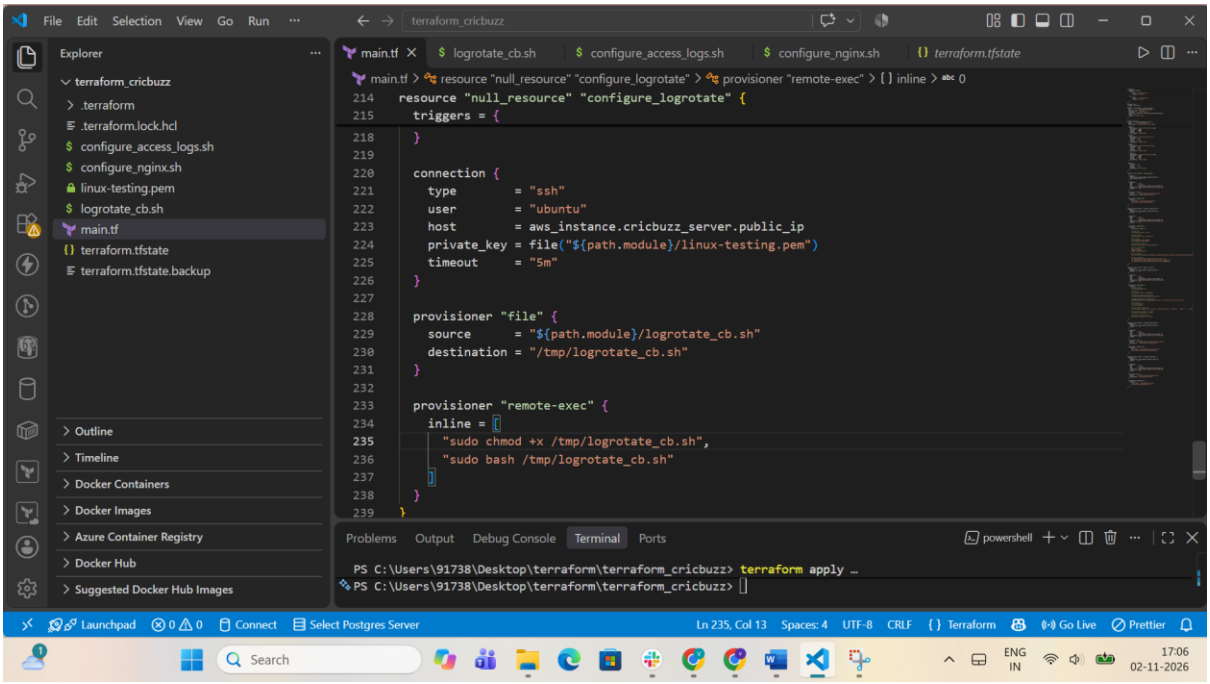
set -e

# Create Logrotate configuration
sudo tee /etc/logrotate.d/cb > /dev/null <<'EOF'
/var/log/cb.log {
    daily
    rotate 7
    compress
    delaycompress
    missingok
    notifempty
    create 0644 root root
    su root adm
}
EOF

echo "Logrotate configuration updated successfully!"
```

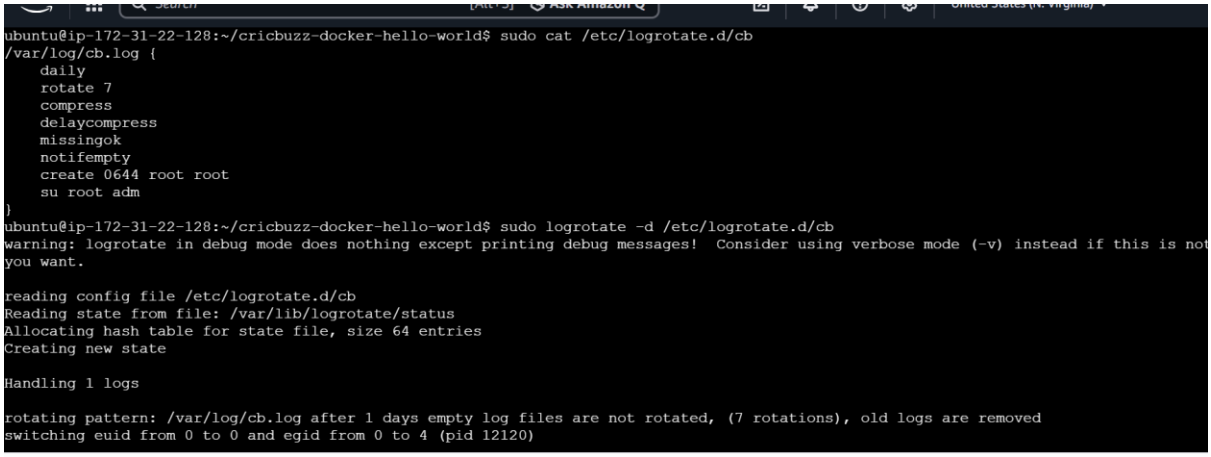
Problems Output Debug Console Terminal Ports powershell

```
PS C:\Users\91738\Desktop\terraform\terraform_cricbuzz> terraform apply --auto-approve
null_resource.configure_logrotate (remote-exec): Logrotate configuration updated successfully!
null_resource.configure_logrotate: Creation complete after 10s [id=952254673564497191]
```

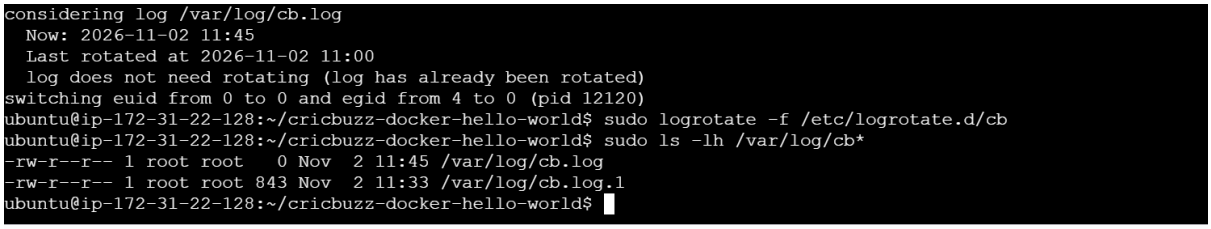



```
PS C:\Users\91738\Desktop\terraform\terraform_cricbuzz> terraform apply --auto-approve
null_resource.configure_logrotate (remote-exec):   Certificate: false
null_resource.configure_logrotate (remote-exec):   SSH Agent: false
null_resource.configure_logrotate (remote-exec):   Checking Host Key: false
null_resource.configure_logrotate (remote-exec):   Target Platform: unix
null_resource.configure_logrotate (remote-exec): Connected!
null_resource.configure_logrotate (remote-exec): Logrotate configuration created successfully!
null_resource.configure_logrotate: Creation complete after 9s [id=2204308499957078938]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
PS C:\Users\91738\Desktop\terraform\terraform_cricbuzz>
```



i-01826e31ffea789e7 (Cricbuzz-Server)



i-01826e31ffea789e7 (Cricbuzz-Server)